

面向历史起终点查询负载的无路径监督精确最短路压缩索引学习方法研究计划书

版本：v2.0

定位：可直接作为立项、开题、组会讨论和原型实现的研究计划

核心原则：不使用历史最短路径标签，不让神经网络直接预测路径，最终查询保持精确最短路正确性。

1. 研究题目

1.1 中文题目

面向历史起终点查询负载的无路径监督精确最短路压缩索引学习方法

1.2 英文暂定题目

Learning Exact Shortest-Path Compression Indexes from Origin-Destination Workloads without Path Supervision

1.3 简称

DFCI：需求场压缩索引

其中：

名称	含义
历史起终点查询负载	用户过去查询的起点、终点和时间分布。
无路径监督	训练时不使用传统最短路算法生成的路径、边经过次数或中转点标签。
压缩索引	提前保存局部区域的精确距离信息，使在线查询少搜索一部分图。
精确最短路	查询结果必须与原图真实最短路完全一致，而不是近似结果。

2. 研究背景

最短路查询是地图导航、校园导航、物流配送、机器人移动、游戏寻路和园区交通系统中的基础问题。传统方法包括 Dijkstra、A*、双向搜索、分层预处理、收缩层次、标签索引、树分解索引等。这些方法通常主要依赖道路图本身的拓扑结构和边权信息。

然而，在真实系统中，查询请求往往并不是均匀随机出现的。以校园导航为例：早上大量查询可能从宿舍区到教学楼，中午大量查询可能从教学楼到食堂，晚上大量查询可能从图书馆到宿舍区。也就是说，真实查询中存在明显的热点区域、高频起终点对和时间规律。

这些历史查询负载说明：用户并不是“平均地使用整张图”，而是在反复查询某些区域之间的路径。如果能够利用这种负载规律，为未来查询提前组织更合适的索引结构，就有可能在保持精确性的前提下降低平均查询成本。

本研究关注的问题是：

能否只根据历史起点、终点和时间信息，学习出一种适合当前查询负载的精确最短路压缩索引，而不提前使用传统算法生成的历史路径作为监督标签？

3. 已有工作风险与研究边界

3.1 必须正视的已有工作

经过重新审阅，原始想法中“利用历史查询负载改善最短路查询结构”这一大方向已经存在高度相关工作。因此，本研究不能简单声称“首次使用历史负载加速最短路”。

需要重点关注的相关工作包括：

1. **Workload-Aware Shortest Path Distance Querying in Road Networks, ICDE 2022**。该工作已经研究了面向查询负载的路网最短距离查询，并提出负载感知的索引结构。
2. **Reinforcement Learning based Tree Decomposition for Distance Querying in Road Networks, ICDE 2023**。该工作将强化学习用于路网距离查询中的树分解结构优化。
3. **Neural A* Search, ICML 2021**。该方向使用神经网络生成搜索引导，并通过专家路径监督训练，属于学习式路径规划的重要代表。
4. **Time-dependent road network shortest path querying with shortcuts, 2023**。该方向研究在时变路网和树结构上选择快捷连接，以提高最短路查询效率。

因此，本研究必须避开以下较弱表述：

- 首次利用历史查询负载加速最短路；
- 首次使用学习方法构造最短路索引；
- 首次将神经网络与经典最短路算法结合；
- 首次通过快捷边加速路网查询。

这些表述容易被已有工作覆盖。

3.2 本研究的准确边界

本研究应当将创新点收缩到更明确的位置：

本研究只使用未经传统最短路算法加工的历史起终点查询记录，学习在固定存储预算下哪些连通区域值得被精确压缩；压缩表由精确算法物化，查询结果可证明等于原图最短路。

换句话说，本研究不是研究“神经网络直接求最短路”，而是研究：

神经网络如何在不看历史路径答案的情况下，为经典精确算法生成更适合当前查询负载的索引结构。

4. 研究目标

本项目的目标是设计、实现并验证一种新的学习式最短路索引框架，满足以下要求。

4.1 理论目标

- 明确提出一种无路径监督的历史查询负载建模方式。
- 设计一种面向起点需求和终点需求的双向需求场传播网络。
- 设计一种沿真实道路图生长的连通区域生成方法。
- 设计一种在固定存储预算下选择压缩区域的机制。
- 证明压缩索引不会破坏最短路正确性。

4.2 工程目标

- 实现基础 Dijkstra 和双向 Dijkstra 查询器。
- 实现区域压缩索引构建流程。
- 实现混合图精确查询流程。
- 实现历史起终点查询负载生成器。
- 实现需求场传播网络和区域价值评估模型。
- 与基础算法、人工规则和相关强基线进行对比。

4.3 实验目标

- 在热点负载下减少平均展开节点数和查询时间。
- 在冷门查询上避免严重退化。
- 在相同存储预算下优于随机区域压缩和简单热度规则。
- 通过消融实验证明双向需求场、连通区域和预算选择确实有效。

5. 问题定义

给定一张带权图：

$$G = (V, E, w)$$

其中：

- G 表示原始道路图；
- V 表示节点集合，例如路口、建筑入口或道路端点；
- E 表示边集合，例如两点之间可通行的道路；
- w 表示边权函数；
- $w(u, v)$ 表示从节点 u 到节点 v 的道路代价，可以是距离、时间或综合成本。

边权满足：

$$w(u, v) > 0$$

这表示每条道路的代价为正数，从而保证 Dijkstra 等精确最短路算法适用。

历史查询记录表示为：

$$\mathcal{Q} = \{(s_i, t_i, \tau_i)\}_{i=1}^M$$

其中：

- $\mathcal{Q}$ 表示历史查询集合；
- M 表示历史查询总数；
- s_i 表示第 i 次查询的起点；
- t_i 表示第 i 次查询的终点；
- τ_i 表示第 i 次查询发生的时间段。

需要特别强调的是， $\mathcal{Q}$ 中不包含任何路径标签。也就是说，其中不包含：

- 最短路径经过的节点序列；
- 最短路径经过的边序列；
- 某条边被历史最短路经过的次数；
- 某个节点作为历史最短路中转点出现的次数；
- 任意起终点对的最短距离标签。

本研究要学习一个索引生成器：

$$I_\theta = F_\theta(G, \mathcal{Q})$$

其中：

- F_θ 表示带参数的学习模型；
- θ 表示模型参数；
- I_θ 表示由模型生成的压缩索引。

对任意在线查询 (s, t) ，使用索引后的查询结果应满足：

$$d_{I_\theta}(s, t) = d_G(s, t)$$

其中：

- $d_G(s, t)$ 表示原图中的真实最短距离；
- $d_{I_\theta}(s, t)$ 表示使用索引后得到的距离。

这表示索引只能加速，不能改变答案。

6. 不做什么

为了保持研究的纯粹性，本项目明确不做以下几类事情。

6.1 不使用历史最短路径作为监督标签

本研究不先对历史查询运行 Dijkstra 或 A*，再统计历史路径经过的节点和边。原因是这些信息已经包含传统算法的答案。如果网络学习这些统计量，本质上就是学习传统算法留下的痕迹，而不是从原始用户行为中发现结构。

6.2 不让神经网络直接预测路径

本研究不让网络直接输出最短路径、下一跳、路径长度或路径边集合。最终路径仍由精确算法和可还原的压缩表保证。

6.3 不把普通节点聚类当作核心方案

本研究不采用简单的“节点特征、图神经网络、节点向量、KMeans 聚类”路线。普通聚类可能把图上不连通的节点分到同一区域，难以构建可证明正确的区域压缩表。

6.4 不优先研究算法调度

原始设想中曾考虑对子图分别使用 Floyd、Dijkstra、A* 等不同算法。新版计划暂时不把这作为主线，因为它会引入过多变量，使实验难以判断收益到底来自区域压缩、网络结构还是算法调度。第一阶段固定使用精确查询器，重点验证学习式索引结构是否有效。

7. 方法总览

本研究提出一种“需求场压缩索引”框架。整体流程如下：



其中，神经网络负责选择和组织结构，精确算法负责计算和保证正确性。

8. 输入特征设计

8.1 节点静态特征

每个节点 v 的静态特征记为 g_v ，可以包括：

- 节点坐标；
- 节点度数；
- 相邻边平均长度；
- 相邻边最短长度；
- 相邻边最长长度；
- 道路类型；
- 是否处于基础网格边界；
- 是否为道路端点。

节点度数指与节点直接相连的边的数量。例如，一个十字路口通常有四条道路相连，因此度数可能为 4。

8.2 起点频率

将一天划分为 R 个时间段。节点 v 在时间段 r 中作为起点的频率定义为：

$$o_v^{(r)} = \frac{\sum_{i=1}^M \mathbf{1}[s_i = v, \tau_i = r]}{\sum_{i=1}^M \mathbf{1}[\tau_i = r] + \varepsilon}$$

其中：

- $o_v^{(r)}$ 表示节点 v 在时间段 r 作为起点的比例；
- $\mathbf{1}[\cdot]$ 是判断函数，条件成立时取 1，否则取 0；
- ε 是很小的正数，用于避免分母为 0。

8.3 终点频率

节点 v 在时间段 r 中作为终点的频率定义为：

$$d_v^{(r)} = \frac{\sum_{i=1}^M \mathbf{1}[t_i = v, \tau_i = r]}{\sum_{i=1}^M \mathbf{1}[\tau_i = r] + \varepsilon}$$

其中， $d_v^{(r)}$ 表示节点 v 在时间段 r 作为终点的比例。

8.4 起终点区域关系

只知道单个节点是否热门还不够。还需要知道哪些区域之间存在高频查询。

先把地图粗略划分为若干基础片区。基础片区只是输入摘要工具，不是最终索引区域。设基础片区 a 到基础片区 b 在时间段 r 中的查询比例为：

$$m_{ab}^{(r)} = \frac{\sum_{i=1}^M \mathbf{1}[s_i \in a, t_i \in b, \tau_i = r]}{\sum_{i=1}^M \mathbf{1}[\tau_i = r] + \varepsilon}$$

其中， $m_{ab}^{(r)}$ 表示在时间段 r 中，从片区 a 查询到片区 b 的频率。

8.5 节点总输入

最终，每个节点的输入表示为：

$$x_v = [g_v, o_v^{(1)}, \dots, o_v^{(R)}, d_v^{(1)}, \dots, d_v^{(R)}, z_v]$$

其中：

- x_v 表示节点 v 的完整输入特征；
- z_v 表示节点所在基础片区的起终点关系摘要；
- 方括号表示把多个特征拼接成一个向量。

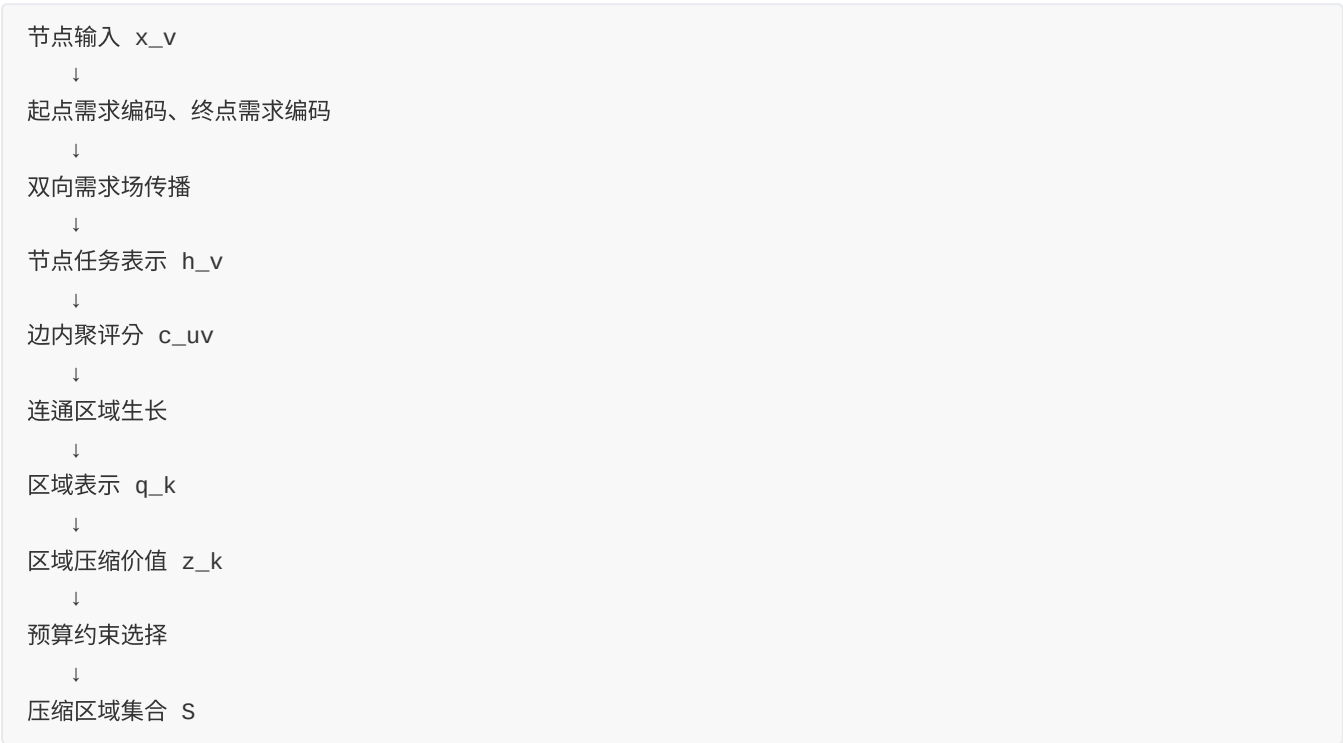
9. 网络结构设计

9.1 总体结构

网络包含四个主要模块：

1. 双向需求场传播模块；
2. 连通区域生成模块；
3. 区域压缩价值评估模块；
4. 预算选择模块。

其结构可以概括为：



9.2 双向需求场传播模块

起点需求和终点需求的含义不同。一个地方可能是热门起点，也可能是热门终点，还可能本身并不热门，却位于高频起终点之间。因此，本研究分别设置起点需求状态和终点需求状态。

定义：

$$O_v^{(\ell)}$$

表示第 ℓ 层时，节点 v 的起点需求状态；定义：

$$D_v^{(\ell)}$$

表示第 ℓ 层时，节点 v 的终点需求状态。

初始状态为：

$$O_v^{(0)} = f_O(x_v)$$

$$D_v^{(0)} = f_D(x_v)$$

其中：

- f_O 是起点需求编码函数；
- f_D 是终点需求编码函数；
- 二者都是可训练函数，可以用小型多层感知机实现。

对于相邻节点 u 和 v ，起点需求从 u 传播到 v 的评分为：

$$e_{u \rightarrow v}^{(\ell)} = f_e([O_u^{(\ell)}, O_v^{(\ell)}, w(u, v), x_u, x_v])$$

其中， f_e 是可训练评分函数。接着将评分归一化为影响比例：

$$\alpha_{u \rightarrow v}^{(\ell)} = \frac{\exp(e_{u \rightarrow v}^{(\ell)})}{\sum_{z \in N(v)} \exp(e_{z \rightarrow v}^{(\ell)})}$$

其中， $N(v)$ 表示节点 v 的邻居集合。

节点 v 从邻居聚合到的新起点需求信息为：

$$\widehat{O}_v^{(\ell+1)} = \sum_{u \in N(v)} \alpha_{u \rightarrow v}^{(\ell)} \Phi_O(O_u^{(\ell)}, w(u, v))$$

其中， Φ_O 是可训练的信息变换函数。

随后用门控方式更新：

$$O_v^{(\ell+1)} = g_v^{(\ell)} \odot \widehat{O}_v^{(\ell+1)} + (1 - g_v^{(\ell)}) \odot O_v^{(\ell)}$$

其中：

- $g_v^{(\ell)}$ 是门控系数，取值在 0 到 1 之间；

- $\odot$ 表示逐元素相乘；
- 门控系数越大，节点越相信邻居传来的新信息；
- 门控系数越小，节点越保留自己的旧信息。

终点需求传播采用类似形式，但语义是“目的地影响向周围扩散”。更新为：

$$D_v^{(\ell+1)} = \bar{g}_v^{(\ell)} \odot \widehat{D}_v^{(\ell+1)} + (1 - \bar{g}_v^{(\ell)}) \odot D_v^{(\ell)}$$

其中：

$$\widehat{D}_v^{(\ell+1)} = \sum_{u \in N(v)} \beta_{u \rightarrow v}^{(\ell)} \Phi_D(D_u^{(\ell)}, w(u, v))$$

最终，经过 L 层传播后，节点表示为：

$$h_v = [x_v, O_v^{(L)}, D_v^{(L)}, O_v^{(L)} \odot D_v^{(L)}, |O_v^{(L)} - D_v^{(L)}|]$$

其中：

- h_v 是节点 v 的最终任务表示；
- $O_v^{(L)} \odot D_v^{(L)}$ 表示起点需求和终点需求同时强的位置；
- $|O_v^{(L)} - D_v^{(L)}|$ 表示起点需求与终点需求的差异。

9.3 连通区域生成模块

区域必须在原图中连通。对于边 (u, v) ，网络计算它适合保留在区域内部的程度：

$$c_{uv} = \sigma(f_C([h_u, h_v, w(u, v)]))$$

其中：

- c_{uv} 表示边 (u, v) 的内聚程度；
- f_C 是可训练评分函数；
- σ 是压缩函数：

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

若 c_{uv} 接近 1，说明网络倾向于让 u 和 v 属于同一区域；若 c_{uv} 接近 0，说明这条边更像区域边界。

网络为每个节点计算成为区域种子的倾向：

$$r_v = \sigma(f_R(h_v))$$

其中， r_v 表示节点 v 成为区域核心的概率。选择若干种子后，区域沿真实道路边生长。

设 $P_{vk}^{(\ell)}$ 表示第 ℓ 轮时节点 v 属于区域 k 的程度。更新公式为：

$$\widetilde{P}_{vk}^{(\ell+1)} = \gamma P_{vk}^{(\ell)} + \sum_{u \in N(v)} c_{uv} P_{uk}^{(\ell)}$$

其中：

- $\widetilde{P}_{vk}^{(\ell+1)}$ 是未归一化的区域归属评分；
- γ 是保留原有归属的系数；
- $c_{uv}P_{uk}^{(\ell)}$ 表示区域归属从邻居 u 沿边 (u, v) 传播到 v 。

然后归一化：

$$P_{vk}^{(\ell+1)} = \frac{\exp(\widetilde{P}_{vk}^{(\ell+1)}/T)}{\sum_{j=1}^K \exp(\widetilde{P}_{vj}^{(\ell+1)}/T)}$$

其中：

- K 是区域数量；
- T 是温度参数；
- T 越小，节点越倾向于明确属于某一个区域。

最后，节点被分配到概率最大的区域：

$$A(v) = \arg \max_k P_{vk}$$

由于软生长不能百分百保证区域连通，还需要进行连通性修复。修复规则为：检查每个区域的连通块，保留最大连通块，将孤立碎片重新分配给相邻且连接最强的区域。

修复比例定义为：

$$\rho_{repair} = \frac{\text{被重新分配的节点数}}{|V|}$$

该指标用于评估区域生成质量。若修复比例过高，说明模型生成区域的能力不足。

9.4 区域边界定义

对区域 R_k ，其边界节点集合定义为：

$$B_k = \{v \in R_k \mid \exists u \notin R_k, (v, u) \in E\}$$

其中：

- R_k 是第 k 个区域；
- B_k 是区域边界节点集合；
- 边界节点表示从该区域进出外部时可能经过的节点。

本研究要求所有真实边界节点都必须保留。网络可以决定某个区域是否值得压缩，但不能为了节省空间而随意删掉真实出口。否则可能漏掉全局最短路，从而破坏精确性。

9.5 区域压缩价值评估模块

对每个区域构造区域表示：

$$q_k = [\text{Pool}(\{h_v \mid v \in R_k\}), |R_k|, |B_k|, L_k]$$

其中：

- q_k 是区域 R_k 的整体表示；
- Pool 是区域内节点表示的汇总函数；
- $|R_k|$ 是区域内部节点数；
- $|B_k|$ 是区域边界节点数；
- L_k 是区域相关的原始查询负载。

区域负载 L_k 只能根据原始起点和终点查询计算，可以包含：

- 起点位于该区域的查询比例；
- 终点位于该区域的查询比例；
- 该区域与其他区域之间的查询频率；
- 不同时间段下该区域的负载变化。

区域压缩价值定义为：

$$z_k = \sigma(f_I(q_k))$$

其中， z_k 越大，表示区域 R_k 越值得建立压缩索引。

9.6 预算选择模块

压缩一个区域需要保存边界节点之间的精确距离表。如果区域 R_k 有 $|B_k|$ 个边界节点，那么需要保存的边界距离规模大约与 $|B_k|^2$ 成正比。

区域压缩成本定义为：

$$\text{Cost}(R_k) = \mu |B_k|^2$$

其中， μ 是单条距离记录的空间系数。

在总预算 B 下，选择压缩区域集合 $\mathcal{S}$ ：

$$\max_{\mathcal{S}} \sum_{R_k \in \mathcal{S}} z_k$$

满足：

$$\sum_{R_k \in \mathcal{S}} \text{Cost}(R_k) \leq B$$

这表示在有限存储空间下，选择价值最高的一批区域进行压缩。

10. 精确压缩索引构建

对于被选择压缩的区域 R_k ，计算所有边界节点之间的区域内部最短距离：

$$T_k(a, b) = d_{G[R_k]}(a, b), \quad a, b \in B_k$$

其中：

- $T_k(a, b)$ 是区域 R_k 内，从边界节点 a 到边界节点 b 的精确最短距离；

- $G[R_k]$ 表示只包含区域 R_k 内部节点和内部边的子图；
- $d_{G[R_k]}(a, b)$ 表示在该子图中的真实最短距离。

同时保存路径还原信息：

$$W_k(a, b)$$

其中， $W_k(a, b)$ 表示从 a 到 b 的区域内部路径如何展开回原图中的真实边序列。

这里可以使用 Dijkstra 或其他精确算法计算区域内部距离。它不是训练标签，而是索引物化步骤。两者区别如下：

做法	是否允许	原因
先用 Dijkstra 给历史查询算路径，再让网络学习边经过次数	不允许	这等于把传统算法答案交给网络学习。
网络先选择压缩区域，再用 Dijkstra 计算该区域边界距离表	允许	这是索引构建和正确性保证，不是路径监督。

11. 在线查询与正确性证明

11.1 混合查询图

在线查询时，构造混合查询图 H_I 。它包含：

1. 未压缩区域中的原始边；
2. 压缩区域的边界节点；
3. 压缩区域边界节点之间的精确距离边；
4. 起点和终点所在区域的必要局部连接。

跨区域查询时，搜索可以跳过压缩区域的大量内部节点，只在边界层面进行转移。

11.2 查询流程

给定查询 (s, t) ：

1. 判断 s 和 t 所在区域；
2. 如果二者位于同一个未压缩小区域，直接运行局部精确查询；
3. 如果二者跨区域，则连接起点到所在区域边界，在混合图上搜索，再连接目标区域边界到终点；
4. 得到最短距离；
5. 若需要完整路径，则根据保存的 $W_k(a, b)$ 展开压缩边。

11.3 正确性证明

需要证明：

$$d_{H_I}(s, t) = d_G(s, t)$$

第一，混合图中的每一条压缩边都对应原图中的一条真实路径，且权重等于该真实路径的长度。因此，混合图不会凭空制造比原图更短的假路径。所以：

$$d_G(s, t) \leq d_{H_I}(s, t)$$

第二，压缩区域保留所有真实边界节点，并用边界距离表表示区域内部任意两个边界节点之间的最短通行方式。因此，原图中的任意跨区域路径都可以在混合图中用不更大的代价表示。所以：

$$d_{H_I}(s, t) \leq d_G(s, t)$$

两式合并得到：

$$d_{H_I}(s, t) = d_G(s, t)$$

因此，索引不会改变最短路答案。

12. 训练目标

训练不使用路径标签，而使用真实查询开销作为反馈。

12.1 查询开销

对查询 q ，定义查询开销：

$$C_{query}(q, I_\theta) = n_{settle}(q, I_\theta) + \lambda_r n_{relax}(q, I_\theta)$$

其中：

- n_{settle} 是查询过程中被确认最短距离的节点数量；
- n_{relax} 是查询过程中被检查或松弛的边数量；
- λ_r 是边检查成本的权重。

确认节点和检查边的数量比真实运行时间更稳定，适合作为训练阶段的反馈指标。最终实验仍然需要报告实际查询时间。

12.2 存储成本

索引存储成本定义为：

$$C_{store}(I_\theta) = \sum_{R_k \in \mathcal{S}} \mu |B_k|^2$$

其中， $\mathcal{S}$ 是被压缩区域集合。

12.3 构建成本

索引构建成本记为：

$$C_{build}(I_\theta)$$

它表示建立区域边界距离表所需时间。第一版可以直接记录实际构建时间，也可以用区域规模和边界数量估计。

12.4 冷门查询保护

为了避免模型只服务热点查询，训练查询分布设置为：

$$\mathcal{D}_{train} = (1 - \eta)\mathcal{D}_{history} + \eta\mathcal{D}_{cold}$$

其中：

- $\mathcal{D}_{history}$ 表示按历史频率采样的查询；
- $\mathcal{D}_{cold}$ 表示冷门查询或随机查询；
- η 表示冷门查询比例。

例如设置 $\eta = 0.2$ ，表示 80% 的训练查询来自历史负载，20% 来自冷门补充。

12.5 总损失函数

总训练目标为：

$$\mathcal{L}(\theta) = \mathbb{E}_{q \sim \mathcal{D}_{train}}[C_{query}(q, I_\theta)] + \lambda_s C_{store}(I_\theta) + \lambda_b C_{build}(I_\theta) + \lambda_t C_{tail}(I_\theta)$$

其中：

- $\mathcal{L}(\theta)$ 是要最小化的总损失；
- $\mathbb{E}$ 表示平均；
- C_{tail} 表示最慢一部分查询的平均开销；
- $\lambda_s, \lambda_b, \lambda_t$ 是权重参数。

该目标要求模型在查询速度、索引空间、构建成本和尾部性能之间取得平衡。

13. 训练策略

13.1 阶段一：需求场预训练

先让模型学习历史查询负载的空间结构。可以随机遮住部分节点的起点频率或终点频率，要求模型根据图结构和邻居信息恢复它们。

这一阶段不使用任何路径标签。

13.2 阶段二：固定区域价值学习

先不让模型生成区域，而是使用固定网格或人工区域，训练模型判断哪些区域值得压缩。

目标是验证：即使区域固定，仅根据原始起终点负载学习压缩价值是否有效。

13.3 阶段三：连通区域生成

加入边内聚评分、区域种子选择和区域生长，使模型生成连通区域。此阶段重点观察：

- 区域是否连通；
- 修复比例是否较低；

- 生成区域是否比固定网格更适合压缩；
- 边界规模是否可控。

13.4 阶段四：真实查询反馈优化

模型生成索引后，真实构建压缩表并运行一批训练查询。根据查询展开节点数、检查边数、索引大小和构建成本优化模型。

由于区域选择是离散决策，可以采用：

- 策略梯度；
- 局部替换搜索；
- 结构扰动搜索；
- 先软选择再逐渐硬化的退火策略。

14. 实验设计

14.1 数据集

第一阶段建议使用三类数据：

数据类型	用途
校园路网	方便可视化和解释。
城市局部路网	验证方法在真实道路结构上的效果。
合成路网	控制变量，测试不同热点分布。

校园路网可以从 OpenStreetMap 或自行构造的简化道路图中获得。

14.2 查询负载

设计四种查询负载：

负载类型	说明	目的
均匀负载	所有区域查询概率接近	检查无明显热点时是否退化。
单热点负载	大量查询集中在一组起终点区域	验证模型能否利用明显热点。
多热点负载	多组高频起终点同时存在	验证模型能否处理复杂需求。
时间漂移负载	训练与测试的热点发生变化	验证模型更新和泛化能力。

14.3 对比方法

基础对比方法：

- 1. 原始 Dijkstra；
- 2. 双向 Dijkstra；
- 3. A*；
- 4. 随机区域压缩；
- 5. 固定网格区域压缩；
- 6. 按起点热度选择区域；
- 7. 按终点热度选择区域；
- 8. 按区域查询总量选择区域。

强相关对比方法：

- 1. 负载感知路网最短距离查询方法；
- 2. 强化学习树分解距离查询方法；
- 3. 经典路网索引方法；
- 4. 学习式近似距离方法。

与近似方法比较时，必须单独报告误差；本研究自身要求精确率为 100%。

14.4 评价指标

指标	含义
平均查询时间	一般查询速度。
最慢 5% 查询时间	尾部性能，反映是否存在卡顿查询。
展开节点数	搜索过程中确认的节点数量。
检查边数	搜索过程中检查或松弛的边数量。
索引大小	额外存储空间。
索引构建时间	预处理耗时。
更新成本	查询负载变化后的重新构建或局部更新成本。
正确率	查询结果与原图 Dijkstra 是否一致。

正确率必须达到 100%。

14.5 消融实验

需要进行以下消融：

- 1. 去掉起点需求场；
- 2. 去掉终点需求场；

3. 去掉起点与终点交互项；
4. 不学习区域，改用固定网格；
5. 不考虑边界规模惩罚；
6. 不加入冷门查询训练；
7. 不进行连通性修复；
8. 用随机选择区域代替网络评分；
9. 用简单热度规则代替网络；
10. 只添加快捷边，不进行区域压缩。

如果移除某个模块后效果明显下降，说明该模块确实有贡献。

15. 第一阶段实现计划

15.1 版本 0：纯算法原型

目标：不使用神经网络，先验证区域压缩索引是否可行。

任务：

1. 实现图数据结构；
2. 实现 Dijkstra；
3. 实现双向 Dijkstra；
4. 实现人工区域划分；
5. 识别区域边界节点；
6. 构建边界距离表；
7. 实现混合图查询；
8. 验证查询结果与原图 Dijkstra 一致。

验收标准：

- 查询正确率为 100%；
- 在人工设计的合理区域上，展开节点数下降。

15.2 版本 1：固定区域价值学习

目标：在固定区域上学习哪些区域值得压缩。

任务：

1. 构造历史起终点查询负载；
2. 提取节点和区域特征；
3. 训练区域价值评分模型；
4. 在预算内选择压缩区域；
5. 与随机选择和简单热度规则比较。

验收标准：

- 学习方法优于随机选择；
- 学习方法不弱于简单热度规则；
- 查询正确率为 100%。

15.3 版本 2：双向需求场网络

目标：加入起点需求场和终点需求场传播。

任务：

1. 实现起点需求传播；
2. 实现终点需求传播；
3. 得到节点任务表示；
4. 汇总为区域表示；
5. 训练区域价值模型；
6. 做需求场消融实验。

验收标准：

- 双向需求场优于只使用原始频率；
- 去掉任一方向需求场后性能下降。

15.4 版本 3：连通区域生成

目标：让模型生成连通区域，而不是依赖固定网格。

任务：

1. 实现边内聚评分；
2. 实现区域种子选择；
3. 实现区域沿边生长；
4. 实现连通性修复；
5. 与固定网格区域比较。

验收标准：

- 区域修复比例较低；
- 生成区域在相同预算下优于固定网格；
- 索引大小可控。

15.5 版本 4：真实反馈优化

目标：形成完整训练闭环。

任务：

1. 模型生成区域；
2. 模型选择压缩区域；
3. 构建真实压缩索引；

4. 运行训练查询；
5. 记录查询开销；
6. 通过局部搜索或策略梯度优化模型。

验收标准：

- 热点负载下明显优于基础算法和简单规则；
- 冷门查询没有明显崩溃；
- 消融实验支持核心模块有效。

16. 代码结构建议

推荐代码目录如下：

```
dfci/  
  data/  
    raw/  
    processed/  
  dfci/  
    graph/  
      graph.py  
      dijkstra.py  
      bidirectional_dijkstra.py  
    workload/  
      od_loader.py  
      workload_generator.py  
      feature_builder.py  
    index/  
      partition.py  
      boundary.py  
      compression_table.py  
      hybrid_query.py  
    model/  
      demand_field.py  
      region_grower.py  
      value_estimator.py  
      budget_selector.py  
    train/  
      pretrain_demand.py  
      train_value.py  
      train_feedback.py  
    experiments/  
      run_baselines.py  
      run_ablation.py  
      run_main.py  
    utils/  
      metrics.py  
      visualize.py  
      config.py  
  configs/
```

```
campus.yaml
city_small.yaml
notebooks/
  visualization.ipynb
README.md
```

17. 可视化计划

本项目适合通过可视化展示研究价值。建议展示：

- 1. 原始路网；
- 2. 起点热度图；
- 3. 终点热度图；
- 4. 起点需求场传播结果；
- 5. 终点需求场传播结果；
- 6. 起点与终点需求交汇区域；
- 7. 网络生成的连通区域；
- 8. 被选择压缩的区域；
- 9. 原始 Dijkstra 的搜索范围；
- 10. 使用索引后的搜索范围；
- 11. 不同方法的查询时间对比；
- 12. 负载变化后的索引调整过程。

这些图可以用于组会展示、项目申报、比赛答辩和后续论文插图。

18. 时间安排

时间	任务	产出
第 1 周	阅读相关工作，明确问题定义	文献笔记、问题定义文档
第 2 周	实现基础最短路和区域压缩原型	可运行的版本 0
第 3 周	构造查询负载和特征	负载生成器、特征模块
第 4 周	固定区域价值学习	版本 1 实验结果
第 5 周	实现双向需求场网络	版本 2 模型与消融结果
第 6 周	实现连通区域生成	版本 3 区域可视化
第 7 周	接入真实反馈优化	完整训练闭环
第 8 周	完整实验与报告整理	技术报告、图表、演示材料

19. 风险与应对

19.1 已有工作过于接近

应对策略：

- 不声称首次负载感知；
- 不声称首次学习式最短路索引；
- 强调无路径监督、双向需求场、连通区域压缩和可证明精确性；
- 必须与最接近工作进行对比。

19.2 区域边界过大导致索引爆炸

应对策略：

- 在损失函数中惩罚边界数量；
- 设置最大边界规模；
- 对边界过大的区域禁止压缩；
- 使用固定存储预算作为硬约束。

19.3 查询加速不明显

应对策略：

- 先在热点明显的负载上验证；
- 分析展开节点数和检查边数；
- 可视化搜索范围；
- 检查区域是否覆盖高频查询结构；
- 加入局部替换搜索。

19.4 冷门查询退化

应对策略：

- 在训练中加入冷门查询；
- 报告最慢 5% 查询；
- 保留原图边作为回退；
- 确保正确性不受影响。

19.5 训练成本过高

应对策略：

- 从小图开始；
- 缓存重复区域的边界距离表；
- 每轮只抽样一批查询估计收益；
- 分阶段训练，不一开始就做完整闭环。

20. 最小可行成果

如果完成以下内容，就可以证明本研究初步可行：

- 1. 输入只使用历史起点、终点和时间；
 - 2. 不使用历史最短路径标签；
 - 3. 能构建区域压缩索引；
 - 4. 查询结果与原图 Dijkstra 完全一致；
 - 5. 在热点负载下展开节点数下降；
 - 6. 在相同预算下优于随机区域压缩和简单热度规则。
-

21. 预期论文结构

后续若整理成论文，可以采用以下结构：

- 1. 引言；
 - 2. 相关工作；
 - 3. 问题定义；
 - 4. 无路径监督的需求负载建模；
 - 5. 双向需求场传播网络；
 - 6. 连通区域压缩索引生成；
 - 7. 精确查询与正确性证明；
 - 8. 训练目标与优化方法；
 - 9. 实验设计；
 - 10. 实验结果；
 - 11. 消融分析；
 - 12. 局限性与未来工作；
 - 13. 结论。
-

22. 当前最优先的下一步

当前最应该先做版本 0，而不是直接写神经网络。

版本 0 的目标是验证区域压缩索引本身是否成立：

输入：一张小路网、一个人工区域划分、一批起终点查询
输出：原始 Dijkstra 与区域压缩查询的展开节点数、检查边数和查询时间对比

如果版本 0 没有收益，说明压缩结构本身需要调整。只有版本 0 能稳定减少搜索范围，后续的需求场网络才有意义。

23. 参考文献与相关资料

1. Bolong Zheng 等, **Workload-Aware Shortest Path Distance Querying in Road Networks**, ICDE 2022。
2. Bolong Zheng 等, **Reinforcement Learning based Tree Decomposition for Distance Querying in Road Networks**, ICDE 2023。
3. Ryo Yonetani 等, **Path Planning using Neural A* Search**, ICML 2021。
4. Zengyang Gong 等, **Querying Shortest Path on Large Time-Dependent Road Networks with Shortcuts**, arXiv 2023。
5. Lingkun Wu 等, **Shortest Path and Distance Queries on Road Networks: An Experimental Evaluation**, PVLDB 2012。

24. 审阅记录

本版本相对上一版做了以下修正：

1. 删除了容易被理解为“先用传统算法标路径再让网络学习”的表述。
 2. 将研究重点从“GNN 表征加聚类”调整为“无路径监督的连通区域压缩索引生成”。
 3. 明确区分了“禁止作为训练标签的历史最短路径”和“允许用于物化索引的精确算法”。
 4. 补充了正确性证明，说明压缩索引不会改变最短路答案。
 5. 加入已有工作的风险分析，避免过度声称创新。
 6. 统一了术语，包括“起终点查询负载”“需求场”“压缩区域”“边界节点”“混合查询图”。
 7. 检查并统一了公式符号，避免同一变量多义。
 8. 检查了 Markdown 标题层级、表格格式、代码块和数学公式闭合情况。
-

25. 一句话总结

本研究不让神经网络学习最短路答案，而是让神经网络在不使用历史路径标签的前提下，根据真实起终点查询负载，学习如何在有限存储预算内选择值得精确压缩的连通区域；最终所有压缩结构由精确算法物化，在线查询结果可证明等于原图真实最短路。